NURa Hand-in 2

Silvan Toet (S1827332)

Date

Write a short description of your solution in each section. Explain your choices. Explicitly draw conclusions from your results.

You can refer to selected lines of code where you think it is necessary. However, you must provide the full code relevant to each (sub)question below it.

Don't forget to describe your figures and mention what conclusions you draw from them!

1 Satellite galaxies around a massive central

1.1 (1a) Determining the normalization

Explain the integration method used (e.g. Romberg integration or another Richardson method). Describe how you implemented the spherical volume element dV.

The spherical volume element dV can be defined as follows: We know that the volume can be computed in spherical coordinates using the triple integral

$$\iiint_V n(x)dV = \int_{\phi=0}^{2\pi}\int_{\theta=0}^{\pi}\int_{r=0}^{r_{max}} n(x)r^2\sin\theta\, dr\, d\theta\, d\phi \tag{1}$$

$$= 4\pi A\langle N_{sat}\rangle \int_{x=0}^{x_{max}=5} x^2\left(\frac{x}{b}\right)^{a-3}\exp(-\left(\frac{x}{b}\right)^c)dx = \langle N_{sat}\rangle \tag{2}$$

$$-> A = \frac{1}{4\pi\int_{x=0}^{x_{max}=5} x^2\left(\frac{x}{b}\right)^{a-3}\exp(-\left(\frac{x}{b}\right)^c)dx} \tag{3}$$

Since n(x) where x is the radius relative to virial radius $x = r/r_{vir}$ is only dependent on r, we can easily solve for θ and ϕ. Dividing by $\langle N_{sat}\rangle$ on both sides and rearranging gives us an expression for A.

Next we use Romberg integration (i.e. Richardson extrapolation applied to integration) to numerically integrate the integral in the expression. Romberg integration first applies the extended trapezoid rule for N intervals, then 2N, 4N, ... up to $2^m\times$ N intervals, where m is the order of the initial approximations. Next we use the fact that each successive order's leading error term is exactly a 1/4 times the leading error term of the previous order. Subtracting the two approximations from each other in such a way as to cancel the leading error term results in m-1 new approximations. We can do this subtraction trick m-1 times. Since after each subtraction we no longer need the two values used for this, we can store the initial approximations and all subsequent improvements in a vector of length m. At the end of the algorithm the first element of the vector contains the best estimate of the integral and the best estimate for the error is the

absolute difference between the first two elements. Table 1 shows the value of the normalization constant A to full precision.

Table 1: Normalization constant of the satellite galaxy profile.

Normalization constant A
9.19836296311

1.2 Code for the integrator

```python
# imports
import numpy as np
import matplotlib.pyplot as plt


def n(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : float | ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function


##### Integrator block #####


# Below we provide a template for romberg integration
# You can implement this or use another integration method based on some form of Richardson extrapolation
def romberg_integrator(
    func: callable, bounds: tuple, order: int = 5, err: bool = False, args: tuple = ()
) -> float | tuple[float, float]:
    """
    Romberg integration method

    Parameters
```

```python
    ----------
    func : callable
        Function to integrate.
    bounds : tuple
        Lower- and upper bound for integration.
    order : int, optional
        Order of the integration.
        The default is 5.
    err : bool, optional
        Whether to retun first error estimate.
        The default is False.
    args : tuple, optional
        Arguments to be passed to func.
        The default is ().

    Returns
    -------
    float
        Value of the integral. If err=True, returns the tuple
        (value, err), with err a first estimate of the (relative)
        error.
    """
    # TODO: implement Romberg integration method

    if err:
        return 0.0, 0.0  # (value, error)
    return 0.0


#### Sampler block ####


def sampler(
    dist: callable,
    min: float,
    max: float,
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
    """
    Sample a distribution using sampling method of your choice

    Parameters
    dist : callable
        Distribution to sample
    min :
        Minimum value for sampling
    max : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """

    return np.zeros(Nsamples)

```

```python

#### Sorting block ####


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """
    Sort a 1D array using a sorting algorithm of your choice

    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
        If False, return a sorted copy

    Returns
    -------
    sorted_arr : ndarray
        Sorted array (same shape as arr)

    """
    if inplace:
        sorted_arr = arr
    else:
        sorted_arr = arr.copy()

    # TODO: sort sorted_arr in-place here

    return sorted_arr


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """
    Choose given number of random elements from an array, without replacement

    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    # TODO: Implement your choice function here, e.g. by using Fisher-Yates shuffling
    return arr[:size].copy()


##### Derivative block #####


def dn_dx(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
```

```python
) -> float | np.ndarray:
    """
    Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # TODO: Write the analytical derivative of n(x) here
    return 0.0


def finite_difference(
    function: callable, x: float | np.ndarray, h: float
) -> float | np.ndarray:
    """
    A building block to compute derivative using finite differences

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h : float
        Step size for finite difference

    Returns
    -------
    dy : float | ndarray
        Derivative at x
    """
    # TODO: Implement finite difference method
    return 0.0


def compute_derivative(
    function: callable,
    x: float | np.ndarray,
    h_init: float,
    # For Ridders use parameters below:
    # d: float, # Factor by which to decrease h_init every iteration
    # eps: float, # Relative error
    # max_iters: int = 10, 3 Maximum number of iterations before exiting
) -> float | np.ndarray:
```

```python
    """
    Function to compute derivative

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference

    Returns
    -------
    df : float | ndarray
        Derivative at x
    """
    # TODO: Implement derivative
    return 0.0


def main():

    # Values from the hand-in
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    bounds = (0, 5)
    xmin, xmax = 10**-4, 5
    N_generate = 10000
    xx = np.linspace(xmin, xmax, N_generate)

    integrand = lambda x, a, b, c: 0.0  # insert the correct function
    integral, err = romberg_integrator(
        integrand, bounds, order=2, args=(a, b, c), err=True
    )

    # Normalisation
    A = 1.0  # to be computed
    with open("Calculations/satellite_A.txt", "w") as f:
        f.write(f"{A:.12g}\n")
    integrand = lambda x, a, b, c: 0.0  # replace by the correct function
    integrated_Nsat = (
        0.0  # replace by the correct integral, e.g. by calling your integrator
    )

    p_of_x = (
        lambda x: 0.0
    )  # replace by the normalised distribution of satellite galaxies as a function of x

    # Numerically determine maximum to normalize p(x) for sampling
    pmax = 0.0  # replace by taking the maximum value of p_of_x

    p_of_x_norm = lambda x: 0.0  # replace by the normalised distribution
    random_samples = np.zeros(
        N_generate
    )  # replace by your sampler(p_of_x_norm, min=xmin, max=xmax, Nsamples=N_generate, args=())

    edges = 10 ** np.linspace(np.log10(xmin), np.log10(xmax), 21)
```

```python

    hist = np.histogram(
        xmin + np.sort(np.random.rand(N_generate)) * (xmax - xmin), bins=edges
    )[
        0
    ]  # replace!
    hist_scaled = (
        1e-3 * hist
    )  # replace; this is NOT what you should be plotting, this is just a random example to get a plot with reasonable y values (think about how you *should* scale hist)

    fig = plt.figure()
    relative_radius = edges.copy()  # replace!
    analytical_function = edges.copy()  # replace

    fig1b, ax = plt.subplots()
    ax.stairs(
        hist_scaled, edges=edges, fill=True, label="Satellite galaxies"
    )  # just an example line, correct this!
    plt.plot(
        relative_radius, analytical_function, "r-", label="Analytical solution"
    )  # correct this according to the exercise!
    ax.set(
        xlim=(xmin, xmax),
        ylim=(10 ** (-3), 10),  # you may or may not need to change ylim
        yscale="log",
        xscale="log",
        xlabel="Relative radius",
        ylabel="Number of galaxies",
    )
    ax.legend()
    plt.savefig("Plots/my_solution_1b.png", dpi=600)

    # Cumulative plot of the chosen galaxies (1c)
    chosen = xmin + np.sort(np.random.rand(Nsat)) * (xmax - xmin)  # replace!
    fig1c, ax = plt.subplots()
    ax.plot(chosen, np.arange(100))
    ax.set(
        xscale="log",
        xlabel="Relative radius",
        ylabel="Cumulative number of galaxies",
        xlim=(xmin, xmax),
        ylim=(0, 100),
    )
    plt.savefig("Plots/my_solution_1c.png", dpi=600)

    x_to_eval = 1
    func_to_eval = lambda x: n(x, A, Nsat, a, b, c)
    dn_dx_numeric = 0.0  # replace by your derivative, e.g. compute_derivative(func_to_eval, x_to_eval, h_init=0.1)
    dn_dx_analytic = dn_dx(x_to_eval, A, Nsat, a, b, c)
    with open("Calculations/satellite_deriv_analytic.txt", "w") as f:
        f.write(f"{dn_dx_analytic:.12g}\n")

    with open("Calculations/satellite_deriv_numeric.txt", "w") as f:
        f.write(f"{dn_dx_numeric:.12g}\n")


if __name__ == "__main__":
    main()
```

1.3 (1b) Sampling the satellite distribution

Describe the sampling method used (e.g. rejection sampling). Explain how the random number generator was implemented and how its randomness was tested.

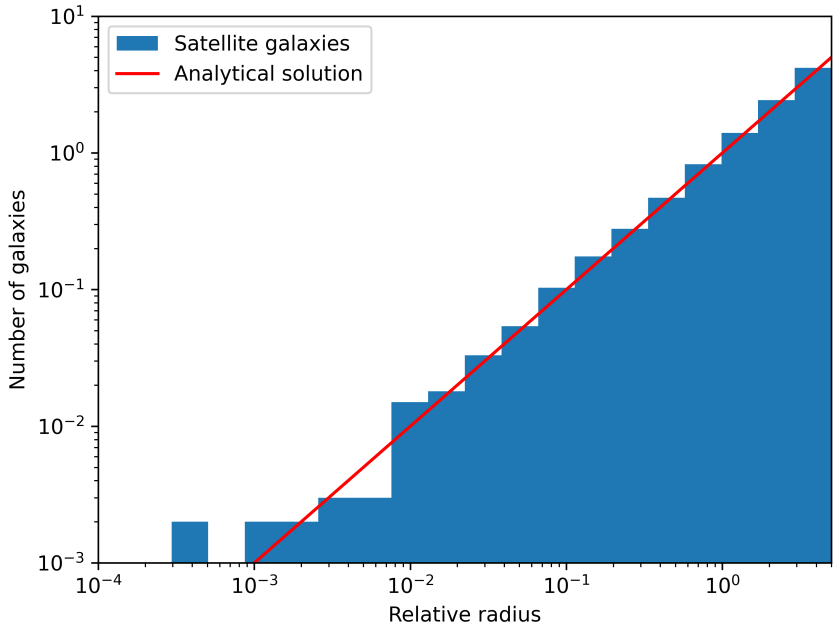

Figure 1: Comparison between the analytical satellite distribution $N(x)$ and the histogram of 10 000 sampled satellite galaxies in logarithmic bins.

Does your sampled distribution agree with the analytical prediction?

1.4 Code for sampling

```python
# imports
import numpy as np
import matplotlib.pyplot as plt


def n(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : float | ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
```

```python
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function


##### Integrator block #####


# Below we provide a template for romberg integration
# You can implement this or use another integration method based on some form of Richardson extrapolation
def romberg_integrator(
    func: callable, bounds: tuple, order: int = 5, err: bool = False, args: tuple = ()
) -> float | tuple[float, float]:
    """
    Romberg integration method

    Parameters
    ----------
    func : callable
        Function to integrate.
    bounds : tuple
        Lower- and upper bound for integration.
    order : int, optional
        Order of the integration.
        The default is 5.
    err : bool, optional
        Whether to retun first error estimate.
        The default is False.
    args : tuple, optional
        Arguments to be passed to func.
        The default is ().

    Returns
    -------
    float
        Value of the integral. If err=True, returns the tuple
        (value, err), with err a first estimate of the (relative)
        error.
    """
    # TODO: implement Romberg integration method

    if err:
        return 0.0, 0.0  # (value, error)
    return 0.0


#### Sampler block ####

```

```python

def sampler(
    dist: callable,
    min: float,
    max: float,
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
    """
    Sample a distribution using sampling method of your choice

    Parameters
    dist : callable
        Distribution to sample
    min :
        Minimum value for sampling
    max : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """

    return np.zeros(Nsamples)


#### Sorting block ####


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """
    Sort a 1D array using a sorting algorithm of your choice

    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
        If False, return a sorted copy

    Returns
    -------
    sorted_arr : ndarray
        Sorted array (same shape as arr)

    """
    if inplace:
        sorted_arr = arr
    else:
        sorted_arr = arr.copy()

    # TODO: sort sorted_arr in-place here
```

```python

    return sorted_arr


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """
    Choose given number of random elements from an array, without replacement

    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    # TODO: Implement your choice function here, e.g. by using Fisher-Yates shuffling
    return arr[:size].copy()


##### Derivative block #####


def dn_dx(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # TODO: Write the analytical derivative of n(x) here
    return 0.0


def finite_difference(
    function: callable, x: float | np.ndarray, h: float
```

```python
) -> float | np.ndarray:
    """
    A building block to compute derivative using finite differences

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h : float
        Step size for finite difference

    Returns
    -------
    dy : float | ndarray
        Derivative at x
    """
    # TODO: Implement finite difference method
    return 0.0


def compute_derivative(
    function: callable,
    x: float | np.ndarray,
    h_init: float,
    # For Ridders use parameters below:
    # d: float, # Factor by which to decrease h_init every iteration
    # eps: float, # Relative error
    # max_iters: int = 10, 3 Maximum number of iterations before exiting
) -> float | np.ndarray:
    """
    Function to compute derivative

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference

    Returns
    -------
    df : float | ndarray
        Derivative at x
    """
    # TODO: Implement derivative
    return 0.0


def main():

    # Values from the hand-in
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    bounds = (0, 5)
    xmin, xmax = 10**-4, 5
    N_generate = 10000
```

```python
    xx = np.linspace(xmin, xmax, N_generate)

    integrand = lambda x, a, b, c: 0.0  # insert the correct function
    integral, err = romberg_integrator(
        integrand, bounds, order=2, args=(a, b, c), err=True
    )

    # Normalisation
    A = 1.0  # to be computed
    with open("Calculations/satellite_A.txt", "w") as f:
        f.write(f"{A:.12g}\n")
    integrand = lambda x, a, b, c: 0.0  # replace by the correct function
    integrated_Nsat = (
        0.0  # replace by the correct integral, e.g. by calling your integrator
    )

    p_of_x = (
        lambda x: 0.0
    )  # replace by the normalised distribution of satellite galaxies as a function of x

    # Numerically determine maximum to normalize p(x) for sampling
    pmax = 0.0  # replace by taking the maximum value of p_of_x

    p_of_x_norm = lambda x: 0.0  # replace by the normalised distribution
    random_samples = np.zeros(
        N_generate
    )  # replace by your sampler(p_of_x_norm, min=xmin, max=xmax, Nsamples=N_generate, args=())

    edges = 10 ** np.linspace(np.log10(xmin), np.log10(xmax), 21)

    hist = np.histogram(
        xmin + np.sort(np.random.rand(N_generate)) * (xmax - xmin), bins=edges
    )[
        0
    ]  # replace!
    hist_scaled = (
        1e-3 * hist
    )  # replace; this is NOT what you should be plotting, this is just a random example to get a plot with reasonable y values (think about how you *should* scale hist)

    fig = plt.figure()
    relative_radius = edges.copy()  # replace!
    analytical_function = edges.copy()  # replace

    fig1b, ax = plt.subplots()
    ax.stairs(
        hist_scaled, edges=edges, fill=True, label="Satellite galaxies"
    )  # just an example line, correct this!
    plt.plot(
        relative_radius, analytical_function, "r-", label="Analytical solution"
    )  # correct this according to the exercise!
    ax.set(
        xlim=(xmin, xmax),
        ylim=(10 ** (-3), 10),  # you may or may not need to change ylim
        yscale="log",
        xscale="log",
        xlabel="Relative radius",
        ylabel="Number of galaxies",
    )
```

```python
    ax.legend()
    plt.savefig("Plots/my_solution_1b.png", dpi=600)

    # Cumulative plot of the chosen galaxies (1c)
    chosen = xmin + np.sort(np.random.rand(Nsat)) * (xmax - xmin)  # replace!
    fig1c, ax = plt.subplots()
    ax.plot(chosen, np.arange(100))
    ax.set(
        xscale="log",
        xlabel="Relative radius",
        ylabel="Cumulative number of galaxies",
        xlim=(xmin, xmax),
        ylim=(0, 100),
    )
    plt.savefig("Plots/my_solution_1c.png", dpi=600)

    x_to_eval = 1
    func_to_eval = lambda x: n(x, A, Nsat, a, b, c)
    dn_dx_numeric = 0.0  # replace by your derivative, e.g. compute_derivative(
    func_to_eval, x_to_eval, h_init=0.1)
    dn_dx_analytic = dn_dx(x_to_eval, A, Nsat, a, b, c)
    with open("Calculations/satellite_deriv_analytic.txt", "w") as f:
        f.write(f"{dn_dx_analytic:.12g}\n")

    with open("Calculations/satellite_deriv_numeric.txt", "w") as f:
        f.write(f"{dn_dx_numeric:.12g}\n")


if __name__ == "__main__":
    main()
```

1.5 (1c) Selecting and sorting satellites

Describe the selection of 100 galaxies (equal probability, no duplicates, no rejection). Explain the sorting algorithm you used.

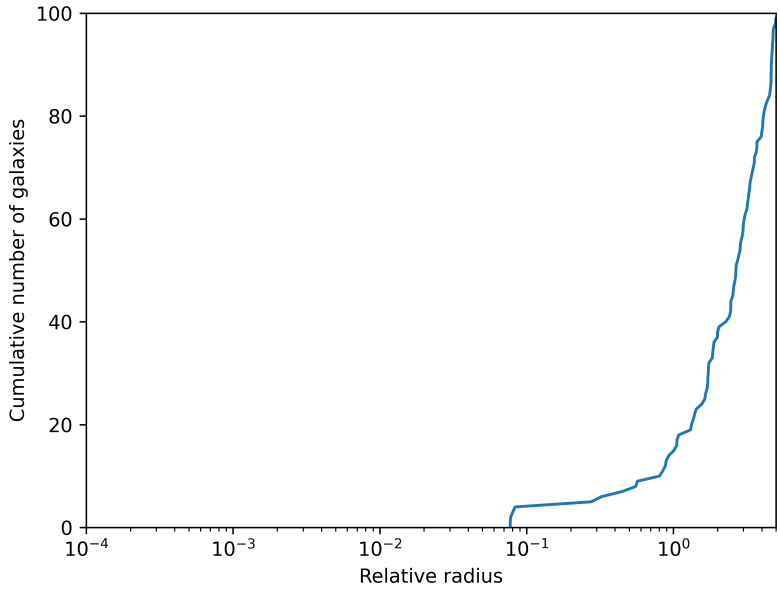

Figure 2: Cumulative number of selected galaxies as a function of relative radius.

1.6 Code for selection and sorting

```python
# imports
import numpy as np
import matplotlib.pyplot as plt


def n(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : float | ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
```

```python
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function


##### Integrator block #####


# Below we provide a template for romberg integration
# You can implement this or use another integration method based on some form of Richardson extrapolation
def romberg_integrator(
    func: callable, bounds: tuple, order: int = 5, err: bool = False, args: tuple = ()
) -> float | tuple[float, float]:
    """
    Romberg integration method

    Parameters
    ----------
    func : callable
        Function to integrate.
    bounds : tuple
        Lower- and upper bound for integration.
    order : int, optional
        Order of the integration.
        The default is 5.
    err : bool, optional
        Whether to retun first error estimate.
        The default is False.
    args : tuple, optional
        Arguments to be passed to func.
        The default is ().

    Returns
    -------
    float
        Value of the integral. If err=True, returns the tuple
        (value, err), with err a first estimate of the (relative)
        error.
    """
    # TODO: implement Romberg integration method

    if err:
        return 0.0, 0.0  # (value, error)
    return 0.0


#### Sampler block ####


def sampler(
    dist: callable,
    min: float,
    max: float,
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
    """
    Sample a distribution using sampling method of your choice

```

```
    Parameters
    dist : callable
        Distribution to sample
    min :
        Minimum value for sampling
    max : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """

    return np.zeros(Nsamples)


#### Sorting block ####


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """
    Sort a 1D array using a sorting algorithm of your choice

    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
        If False, return a sorted copy

    Returns
    -------
    sorted_arr : ndarray
        Sorted array (same shape as arr)

    """
    if inplace:
        sorted_arr = arr
    else:
        sorted_arr = arr.copy()

    # TODO: sort sorted_arr in-place here

    return sorted_arr


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """
    Choose given number of random elements from an array, without replacement

    Parameters
    ----------
    arr : ndarray
```

```
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    # TODO: Implement your choice function here, e.g. by using Fisher-Yates shuffling
    return arr[:size].copy()


##### Derivative block #####


def dn_dx(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # TODO: Write the analytical derivative of n(x) here
    return 0.0


def finite_difference(
    function: callable, x: float | np.ndarray, h: float
) -> float | np.ndarray:
    """
    A building block to compute derivative using finite differences

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h : float
```

```
        Step size for finite difference

    Returns
    -------
    dy : float | ndarray
        Derivative at x
    """
    # TODO: Implement finite difference method
    return 0.0


def compute_derivative(
    function: callable,
    x: float | np.ndarray,
    h_init: float,
    # For Ridders use parameters below:
    # d: float, # Factor by which to decrease h_init every iteration
    # eps: float, # Relative error
    # max_iters: int = 10, 3 Maximum number of iterations before exiting
) -> float | np.ndarray:
    """
    Function to compute derivative

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference

    Returns
    -------
    df : float | ndarray
        Derivative at x
    """
    # TODO: Implement derivative
    return 0.0


def main():

    # Values from the hand-in
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    bounds = (0, 5)
    xmin, xmax = 10**-4, 5
    N_generate = 10000
    xx = np.linspace(xmin, xmax, N_generate)

    integrand = lambda x, a, b, c: 0.0  # insert the correct function
    integral, err = romberg_integrator(
        integrand, bounds, order=2, args=(a, b, c), err=True
    )

    # Normalisation
    A = 1.0  # to be computed
    with open("Calculations/satellite_A.txt", "w") as f:
        f.write(f"{A:.12g}\n")
```

```python
    integrand = lambda x, a, b, c: 0.0  # replace by the correct function
    integrated_Nsat = (
        0.0  # replace by the correct integral, e.g. by calling your integrator
    )

    p_of_x = (
        lambda x: 0.0
    )  # replace by the normalised distribution of satellite galaxies as a function of x

    # Numerically determine maximum to normalize p(x) for sampling
    pmax = 0.0  # replace by taking the maximum value of p_of_x

    p_of_x_norm = lambda x: 0.0  # replace by the normalised distribution
    random_samples = np.zeros(
        N_generate
    )  # replace by your sampler(p_of_x_norm, min=xmin, max=xmax, Nsamples=N_generate, args=())

    edges = 10 ** np.linspace(np.log10(xmin), np.log10(xmax), 21)

    hist = np.histogram(
        xmin + np.sort(np.random.rand(N_generate)) * (xmax - xmin), bins=edges
    )[
        0
    ]  # replace!
    hist_scaled = (
        1e-3 * hist
    )  # replace; this is NOT what you should be plotting, this is just a random example to get a plot with reasonable y values (think about how you *should* scale hist)

    fig = plt.figure()
    relative_radius = edges.copy()  # replace!
    analytical_function = edges.copy()  # replace

    fig1b, ax = plt.subplots()
    ax.stairs(
        hist_scaled, edges=edges, fill=True, label="Satellite galaxies"
    )  # just an example line, correct this!
    plt.plot(
        relative_radius, analytical_function, "r-", label="Analytical solution"
    )  # correct this according to the exercise!
    ax.set(
        xlim=(xmin, xmax),
        ylim=(10 ** (-3), 10),  # you may or may not need to change ylim
        yscale="log",
        xscale="log",
        xlabel="Relative radius",
        ylabel="Number of galaxies",
    )
    ax.legend()
    plt.savefig("Plots/my_solution_1b.png", dpi=600)

    # Cumulative plot of the chosen galaxies (1c)
    chosen = xmin + np.sort(np.random.rand(Nsat)) * (xmax - xmin)  # replace!
    fig1c, ax = plt.subplots()
    ax.plot(chosen, np.arange(100))
    ax.set(
        xscale="log",
        xlabel="Relative radius",
        ylabel="Cumulative number of galaxies",
```

```
        xlim=(xmin, xmax),
        ylim=(0, 100),
    )
    plt.savefig("Plots/my_solution_1c.png", dpi=600)

    x_to_eval = 1
    func_to_eval = lambda x: n(x, A, Nsat, a, b, c)
    dn_dx_numeric = 0.0  # replace by your derivative, e.g. compute_derivative(
    func_to_eval, x_to_eval, h_init=0.1)
    dn_dx_analytic = dn_dx(x_to_eval, A, Nsat, a, b, c)
    with open("Calculations/satellite_deriv_analytic.txt", "w") as f:
        f.write(f"{dn_dx_analytic:.12g}\n")

    with open("Calculations/satellite_deriv_numeric.txt", "w") as f:
        f.write(f"{dn_dx_numeric:.12g}\n")


if __name__ == "__main__":
    main()
```

1.7 (1d) Numerical derivative

Table 2: Derivative of the satellite number density at $x = 1$.

Method	Value
Analytical	0
Numerical	0

Do your analytical and numerical derivatives agree? Why?

1.8 Code for derivative calculation

```
# imports
import numpy as np
import matplotlib.pyplot as plt


def n(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : float | ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

```

```python
    Returns
    -------
    float | ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function


##### Integrator block #####


# Below we provide a template for romberg integration
# You can implement this or use another integration method based on some form of Richardson extrapolation
def romberg_integrator(
    func: callable, bounds: tuple, order: int = 5, err: bool = False, args: tuple = ()
) -> float | tuple[float, float]:
    """
    Romberg integration method

    Parameters
    ----------
    func : callable
        Function to integrate.
    bounds : tuple
        Lower- and upper bound for integration.
    order : int, optional
        Order of the integration.
        The default is 5.
    err : bool, optional
        Whether to retun first error estimate.
        The default is False.
    args : tuple, optional
        Arguments to be passed to func.
        The default is ().

    Returns
    -------
    float
        Value of the integral. If err=True, returns the tuple
        (value, err), with err a first estimate of the (relative)
        error.
    """
    # TODO: implement Romberg integration method

    if err:
        return 0.0, 0.0  # (value, error)
    return 0.0


#### Sampler block ####


def sampler(
    dist: callable,
    min: float,
    max: float,
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
```

```python
    """
    Sample a distribution using sampling method of your choice

    Parameters
    dist : callable
        Distribution to sample
    min :
        Minimum value for sampling
    max : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """

    return np.zeros(Nsamples)


#### Sorting block ####


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """
    Sort a 1D array using a sorting algorithm of your choice

    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
        If False, return a sorted copy

    Returns
    -------
    sorted_arr : ndarray
        Sorted array (same shape as arr)

    """
    if inplace:
        sorted_arr = arr
    else:
        sorted_arr = arr.copy()

    # TODO: sort sorted_arr in-place here

    return sorted_arr


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """
    Choose given number of random elements from an array, without replacement

```

```python
    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    # TODO: Implement your choice function here, e.g. by using Fisher-Yates shuffling
    return arr[:size].copy()


##### Derivative block #####


def dn_dx(
    x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> float | np.ndarray:
    """
    Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # TODO: Write the analytical derivative of n(x) here
    return 0.0


def finite_difference(
    function: callable, x: float | np.ndarray, h: float
) -> float | np.ndarray:
    """
    A building block to compute derivative using finite differences

    Parameters
    ----------
    function : callable
        Function to differentiate
```

```
    x : float | ndarray
        Value(s) to evaluate derivative at
    h : float
        Step size for finite difference

    Returns
    -------
    dy : float | ndarray
        Derivative at x
    """
    # TODO: Implement finite difference method
    return 0.0


def compute_derivative(
    function: callable,
    x: float | np.ndarray,
    h_init: float,
    # For Ridders use parameters below:
    # d: float, # Factor by which to decrease h_init every iteration
    # eps: float, # Relative error
    # max_iters: int = 10, 3 Maximum number of iterations before exiting
) -> float | np.ndarray:
    """
    Function to compute derivative

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference

    Returns
    -------
    df : float | ndarray
        Derivative at x
    """
    # TODO: Implement derivative
    return 0.0


def main():

    # Values from the hand-in
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    bounds = (0, 5)
    xmin, xmax = 10**-4, 5
    N_generate = 10000
    xx = np.linspace(xmin, xmax, N_generate)

    integrand = lambda x, a, b, c: 0.0  # insert the correct function
    integral, err = romberg_integrator(
        integrand, bounds, order=2, args=(a, b, c), err=True
    )

    # Normalisation
```

```python
    A = 1.0  # to be computed
    with open("Calculations/satellite_A.txt", "w") as f:
        f.write(f"{A:.12g}\n")
    integrand = lambda x, a, b, c: 0.0  # replace by the correct function
    integrated_Nsat = (
        0.0  # replace by the correct integral, e.g. by calling your integrator
    )

    p_of_x = (
        lambda x: 0.0
    )  # replace by the normalised distribution of satellite galaxies as a function of x

    # Numerically determine maximum to normalize p(x) for sampling
    pmax = 0.0  # replace by taking the maximum value of p_of_x

    p_of_x_norm = lambda x: 0.0  # replace by the normalised distribution
    random_samples = np.zeros(
        N_generate
    )  # replace by your sampler(p_of_x_norm, min=xmin, max=xmax, Nsamples=N_generate, args=())

    edges = 10 ** np.linspace(np.log10(xmin), np.log10(xmax), 21)

    hist = np.histogram(
        xmin + np.sort(np.random.rand(N_generate)) * (xmax - xmin), bins=edges
    )[
        0
    ]  # replace!
    hist_scaled = (
        1e-3 * hist
    )  # replace; this is NOT what you should be plotting, this is just a random example to get a plot with reasonable y values (think about how you *should* scale hist)

    fig = plt.figure()
    relative_radius = edges.copy()  # replace!
    analytical_function = edges.copy()  # replace

    fig1b, ax = plt.subplots()
    ax.stairs(
        hist_scaled, edges=edges, fill=True, label="Satellite galaxies"
    )  # just an example line, correct this!
    plt.plot(
        relative_radius, analytical_function, "r-", label="Analytical solution"
    )  # correct this according to the exercise!
    ax.set(
        xlim=(xmin, xmax),
        ylim=(10 ** (-3), 10),  # you may or may not need to change ylim
        yscale="log",
        xscale="log",
        xlabel="Relative radius",
        ylabel="Number of galaxies",
    )
    ax.legend()
    plt.savefig("Plots/my_solution_1b.png", dpi=600)

    # Cumulative plot of the chosen galaxies (1c)
    chosen = xmin + np.sort(np.random.rand(Nsat)) * (xmax - xmin)  # replace!
    fig1c, ax = plt.subplots()
    ax.plot(chosen, np.arange(100))
    ax.set(
```

```
        xscale="log",
        xlabel="Relative radius",
        ylabel="Cumulative number of galaxies",
        xlim=(xmin, xmax),
        ylim=(0, 100),
    )
    plt.savefig("Plots/my_solution_1c.png", dpi=600)

    x_to_eval = 1
    func_to_eval = lambda x: n(x, A, Nsat, a, b, c)
    dn_dx_numeric = 0.0  # replace by your derivative, e.g. compute_derivative(
    func_to_eval, x_to_eval, h_init=0.1)
    dn_dx_analytic = dn_dx(x_to_eval, A, Nsat, a, b, c)
    with open("Calculations/satellite_deriv_analytic.txt", "w") as f:
        f.write(f"{dn_dx_analytic:.12g}\n")

    with open("Calculations/satellite_deriv_numeric.txt", "w") as f:
        f.write(f"{dn_dx_numeric:.12g}\n")


if __name__ == "__main__":
    main()
```

2 Heating and cooling in HII regions

2.1 (2a) Photoionization heating and radiative recombination cooling

Describe the implementation of your root-finding method (e.g. bisection, Newton-Raphson).

Table 3: Equilibrium temperature for the simplified heating–cooling model.

Temperature [K]	Absolute error	Relative error
0	0.000e+00	0.000e+00

How many iterations it took? Discuss the convergence of your root finder.

2.2 (2b) More realistic model

Table 4: Equilibrium temperature for different gas densities.

Density n_H [cm^{-3}]	Equilibrium temperature [K]
10^{-4}	0
1	0
10^4	0

Discuss the results. What is the difference for the low, intermediate and high gas density?

2.3 Code for Exercise 2

```
import numpy as np

# Constants (mind the units!)

```

```python
psi = 0.929
Tc = 1e4 # K
Z = 0.015
k = 1.38e-16  # erg/K
aB = 2e-13  # cm^3 / s
A = 5e-10
xi = 1e-15


# There's no need for nH nor ne as they cancel out
def equilibrium1(T, Z, Tc, psi):
    return psi * Tc * k - (0.684 - 0.0416 * np.log(T / (1e4 * Z * Z))) * T * k


def equilibrium2(T, Z, Tc, psi, nH, A, xi, aB):
    return (
        (
            psi * Tc
            - (0.684 - 0.0416 * np.log(T / (1e4 * Z * Z))) * T
            - 0.54 * (T / 1e4) ** 0.37 * T
        )
        * k
        * nH
        * aB
        + A * xi
        + 8.9e-26 * (T / 1e4)
    )


# Derivative function, might be useful if using Newton-Raphson method for root finding
# def equilibrium2_deriv(T, nH):
#     # TODO: Compute derivative of equilibrium2 with respect to T
#     return 0.0


#### root finder ####


def root_finder(
    func: callable,  # add derivative if using Newton-Raphson
    bracket: tuple,
    atol: float = 1e-6,
    rtol: float = 1e-6,
    max_iters: int = 100,
) -> tuple[float, float, float]:
    """
    Find a root of a function

    Parameters
    ----------
    func : callable
        Function to find root of
    bracket : tuple
        Bracket for which to find first secant
    atol : float, optional
        Absolute tolerance.
        The default is 1e-6
    rtol : float, optional
        Relative tolerance.
        The default is 1e-6
    max_iters: int, optional
```

```python
        Maximum number of iterations.
        Teh default is 100

    Returns
    -------
    root : float
        Approximate root
    aerr : float
        Absolute error
    rerr : float
        Relative error
    """
    # TODO: Implement root finder (e.g. bisection, false-position, Newton-Raphson)
    return 0.0, 0.0, 0.0


def main():

    # Initial bracket
    bracket = (1, 1e7)

    root, aerr, rerr = 0.0, 0.0, 0.0  # replace with your root finder

    with open("Calculations/equilibrium_temp_simple.txt", "w") as f:
        f.write(f"{root:.12g} & {aerr:.3e} & {rerr:.3e}")
    #### 2b ####

    # Initial bracket
    bracket = (1, 1e15)

    for nH in [1e-4, 1, 1e4]:

        root, aerr, rerr = 0.0, 0.0, 0.0  # replace with your root finder
        if nH == 1e-4:
            with open("Calculations/equilibrium_low_density.txt", "w") as f:
                f.write(f"{root:.12g}")
        elif nH == 1:
            with open("Calculations/equilibrium_mid_density.txt", "w") as f:
                f.write(f"{root:.12g}")
        elif nH == 1e4:
            with open("Calculations/equilibrium_high_density.txt", "w") as f:
                f.write(f"{root:.12g}")


if __name__ == "__main__":
    main()
```

3 Conclusions